

LLDD BE - Job 2 ImportImpactStore

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจสอบ และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. Overview

รายการ	รายละเอียด
Track	BE
Estimate	19 ชั่วโมง = implementation 14 + unit test 5 (30%)
Owner	Aphiwit <Bank> Khammoon
Target repository	SBP/srm-sps-spsap-store-backend (NestJS + TypeORM · schema sps_store) — batch runner ผัง backend ไม่ผ่าน BFF · cron/พารามิเตอร์อยู่ใน backend config (env/config file)
Objective	นำเข้าคู่ร้านถูกระทบจาก ALLMAP: นำเข้าร้านถูกระทบ-ร้านเปิดใหม่จากวิว ALLMAP เข้า fgi_impact_stores เติมข้อมูลจากตาราง master แล้วใช้กฎ DENY และ ON_PROCESS ตั้งค่า sales_request_status เป็น W / N / P

Common contract reference: ทุกหัวข้อ API/FE ต้องยึด LLDD-BE-API-Common-Contracts.pdf และ LLDD-FE-Integration-Contracts.pdf สำหรับ error/auth/format/pagination/action/RBAC ก่อนลงรายละเอียดเฉพาะหน้าหรือเฉพาะ endpoint

2. Screen / Functional Scope

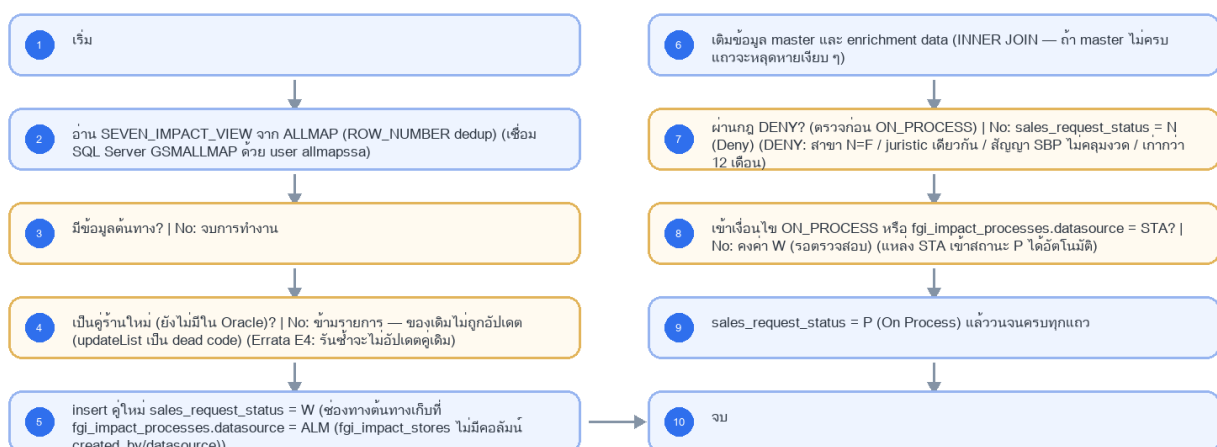
- Main class/script: fgi.main.ImportImpactStore / FGI_ImportImpactStore.sh
- Phase: A
- Output: fgi_impact_stores
- Estimate: 14 ชั่วโมง
- พารามิเตอร์/cron อ่านจาก backend config (config file/env) — ไม่มีตาราง job_configs และไม่มีหน้าจอบทความ (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- Runbook, rerun rule, risk และ history ตามเอกสาร Batch v4.0 · ผลการรันเขียน application log แบบ structured

3. Screenshot Reference

ไม่มีภาพหน้าจอสำหรับหัวข้อนี้ — เป็นเอกสารฝั่ง Backend/Batch ที่ไม่มี UI (ภาพหน้าจอทั้งหมดอยู่ในเอกสารชุด FE)

4. Implementation Flow Diagram (Reference)

LLDD BE - Job 2 ImportImpactStore



รูปที่ 1: Implementation flow reference: LLDD BE - Job 2 ImportImpactStore

5. Field, Format, and Validation

Field / UI	Format	Validation	Behavior
กำหนดการรัน (Cron)	0 07 7 * *	แก้ไขได้	ทุกวันที่ 7 ของเดือน เวลา 07:00
Argument (ขอบเขต งวด)	ALL 2569 06	แก้ไขได้	รูปแบบ ZONES YYYY MM หรือ ALL YYYY MM — ไม่ระบุจะใช้งวดตาม modifyDateToString · □□ ปีในตัวอย่างเป็น พ.ศ. (2569) ตามค่าที่ระบบเดิมใช้กับวิว ALLMAP ซึ่งขัดกับกติกา ค.ศ. ทั้งระบบ (มติ 2026-08-06) — ต้องยืนยันกับเจ้าของ ALLMAP ว่าวิวเก็บปีเป็น พ.ศ. จริงหรือไม่ ถ้าไม่ ให้เปลี่ยนเป็น ค.ศ. (2026)
Source View	allmapssa.SEVEN_IMPACT_VIEW (SQL Server GSMALLMAP)	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	dedup ด้วย ROW_NUMBER
Branch Type ที่เข้าเกณฑ์	B, FAM, FB1, FB2, FC1, FVB, FVC, FPT1	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	FPT1 เข้าเกณฑ์เฉพาะเมื่อ SBP_CANCEL_TYPE_I = 06
กฎ DENY (ตรวจสอบก่อน ON_PROCESS)	สาขา N=F / juristic เดียวกัน / สัญญาไม่คลุมงวด / เก่ากว่า 12 เดือน	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	
PK Sequence	BIGSERIAL ของ fgi_impact_stores (PostgreSQL — ไม่ใช่ named sequence แบบ Oracle)	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	

5.9 Input / Progress / Output Contract

Stage	Contract for implementation
Input	Period year/month, optional zone filter, and ALLMAP SEVEN_IMPACT_VIEW rows.
Progress	query candidate impacted stores, deduplicate by store/month, batch insert impact-store master data, derive related new-store/impact-store records, update verification flags.
Output	FGI_IMPACT_STORE and related impact/new-store tables contain imported candidates for the requested period with duplicate-safe status.

5.90 Job 2 Execution Stages

query candidate impacted stores, deduplicate by store/month, batch insert impact-store master data, derive related new-store/impact-store records, update verification flags.

Order	Service step	Repository	Output / failure contract
1	loadAllmapCandidates	impactStoreRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง
2	resolveImpactProcesses	impactStoreRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง
3	upsertImpactPairs	impactStoreRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง
4	reconcileImportedPairs	impactStoreRepository	คืน metrics และ throw typed error; transaction/rerun ใช้ contract ด้านล่าง

5.91 Job 2 Run Evidence

Evidence	Job-specific value	Acceptance
Input identity	Period year/month, optional zone filter, and ALLMAP SEVEN_IMPACT_VIEW rows.	snapshot input file/business key/period in run record
Output identity	FGI_IMPACT_STORE and related impact/new-store tables contain imported candidates for the requested period with duplicate-safe status.	reconcile input, success, reject and skipped counts
Dedup proof	UNIQUE(impacted_store_code, new_store_code, impact_month); rerun อัปเดตค่าที่เปลี่ยนแต่ไม่สร้างคู่ร้านซ้ำ	rerun fixture produces no duplicate target business key
Transaction proof	สร้าง/หา fgi_impact_processes และ upsert candidate ทีละ chunk ใน transaction; chunk fail rollback เฉพาะ chunk	injected failure leaves no partial committed state outside documented boundary
Security proof	ALLMAP connection ใช้ datasource secretRef และ TLS verify-full; job parameter เก็บได้เฉพาะ datasource alias ไม่เก็บ username/password	config/log/error contains no plaintext secret

5.92 Legacy Java Source Reference

Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/main/ImportImpactStore.java	24-186	Legacy main entrypoint for impacted-store import.
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportStoreJdbc.java	30-84, 170-484	Query SEVEN_IMPACT_VIEW and insert/update FGI impact/new-store records.

Line ranges refer to the legacy Java implementation under /Users/bank_mac/gosoft/java/SBP/fcsJar. Use these ranges to preserve business behavior while implementing the target Node job.

5.93 Target Repository and SQL Contract

Contract	Target implementation
Repository	impactStoreRepository
Idempotency / dedup	UNIQUE(impacted_store_code, new_store_code, impact_month); rerun อัปเดตค่าที่เปลี่ยนแต่ไม่สร้างคู่ร้านซ้ำ
Transaction boundary	สร้าง/หา fgi_impact_processes และ upsert candidate ทีละ chunk ใน transaction; chunk fail rollback เฉพาะ chunk
Security	ALLMAP connection ใช้ datasource secretRef และ TLS verify-full; job parameter เก็บได้เฉพาะ datasource alias ไม่เก็บ username/password

Input / candidate query

```
SELECT impacted_store_code, new_store_code, impact_month, distance_km, region_code, zone_code, branch_type
FROM allmap_seven_impact_view
WHERE impact_month = :impact_month
AND (:zone_code IS NULL OR zone_code = :zone_code)
AND distance_km <= CASE
  WHEN region_code = ANY(:bangkok_metro_region_codes) THEN 1.000
  ELSE 2.000
END;
```

Write / upsert query

```
INSERT INTO fgi_impact_stores
  (impact_process_id, impacted_store_code, new_store_code, impact_month, distance_km, updated_at)
VALUES (:impact_process_id, :impacted_store_code, :new_store_code, :impact_month, :distance_km, CURRENT_TIMESTAMP)
ON CONFLICT (impacted_store_code, new_store_code, impact_month)
DO UPDATE SET distance_km = EXCLUDED.distance_km,
  impact_process_id = EXCLUDED.impact_process_id,
  updated_at = CURRENT_TIMESTAMP;
```

5.94 Target Node Implementation

โครงสร้างนี้ระบุ service/repository เฉพาะงานและต้อง implement ตาม SQL, transaction, idempotency และ security contract ด้านบน โดยทุกชั้นต้องคืน metrics สำหรับ reconcile และ run history

```
export async function runLddBeJob2ImportImpactStore(ctx, services) {
  const run = await services.jobRuns.acquire({
    jobNo: "2", period: ctx.period, triggeredBy: ctx.triggeredBy
  });

  try {
    ctx = { ...ctx, runId: run.id, repository: services.impactStoreRepository };
    const step1 = await services.loadAllMapCandidates(ctx, undefined);
    const step2 = await services.resolveImpactProcesses(ctx, step1);
    const step3 = await services.upsertImpactPairs(ctx, step2);
    const step4 = await services.reconcileImportedPairs(ctx, step3);
    const result = step4;
    await services.jobRuns.finish(run.id, "SUCCESS", result.metrics);
    return { runId: run.id, status: "SUCCESS", ...result };
  } catch (error) {
    await services.jobRuns.finish(run.id, "FAILED", {
      errorCode: error.code ?? "JOB_FAILED",
      errorMessage: error.message
    });
    throw error;
  }
}
```

6. Button / User Action Mapping

Action	Trigger	API / Service	Expected Result
รันตามตารางเวลา	CRON	scheduler → runner (job 2)	อ่าน cron/พารามิเตอร์จาก backend config
รันนอกรอบ (manual/rerun)	CLI	CLI/ops runbook → runner (job 2)	guard ไม่ให้รันซ้อนด้วย distributed lock
แก้พารามิเตอร์/เปิด-ปิด job	CONFIG	แก้ backend config แล้ว deploy	ไม่มี endpoint และไม่มีหน้าจอลงความคุม — หน้า Flow Batch Job เป็น reference อย่างเดียว (2026-08-06)
ตรวจผลการรัน	LOG	application log (structured)	ไม่มีตาราง job_run_histories แล้ว · ไฟล์/ACK ดูที่ interface_transactions

7. API Contract

เอกสารฉบับนี้ไม่มี endpoint ของตัวเอง — เป็นสัญญา/งานภายในที่เอกสารอื่นเรียกใช้ (ดูขอบเขตใน 5.90 Endpoint Implementation Contract) · รายการ endpoint ทั้ง 29 เส้นของ SBPGI อยู่ที่ LLDD-API.pdf และ API design

8. Reference DB Mapping (No Database Page Work)

ส่วนนี้เป็นข้อมูลอ้างอิงสำหรับการ implement API/Job เท่านั้น ไม่ใช่งานสร้างหน้า Database, ไม่ใช่งานออกแบบ DB page และไม่ถูกนับเป็น deliverable แยกของ FE/BE

Table / Object	R/W	Usage
fgi_impact_stores	W	insert คู่ร้านกระทบ-ร้านใหม่ / ตั้ง sales_request_status = W · N · P (ตารางนี้ไม่มี created_by — ช่องทางต้นทางอยู่ที่ fgi_impact_processes.datasource)

9. Skeleton Code (Batch Job 2)

9.1 ไฟล์ที่ต้องสร้าง (Job 2)

โครงไฟล์ของ Job 2 (fgi.main.ImportImpactStore เดิม) วางได้ src/batch/sbpgi/ ของ store-backend โดยใช้ convention เดียวกับ module อื่นๆ: inject custom provider DATA_SOURCE แล้วยิง raw SQL, repository ประกาศเป็น factory provider ที่ใช้ token string, entity อยู่ใน src/entities/

หมายเหตุสำคัญ — src/batch/* ทั้งหมดเป็นของใหม่ที่ยังไม่มีใน store-backend: ปัจจุบัน repo ไม่มีโฟลเดอร์ src/batch เลย และแม้จะติดตั้ง @nestjs/schedule ไว้แล้วก็ยังไม่มี @Cron/@Interval แม้แต่จุดเดียว ดังนั้น runner.ts / scheduler.ts / cli.js / job-failure.notifier.ts คือ งานตั้งต้นของ Phase แรก ที่ต้องสร้างเองทั้งหมด พร้อม register ScheduleModule.forRoot() ใน app.module.ts — ไม่ใช่ของเดิมที่ reuse ได้

Path	หน้าที่
src/batch/sbpgi/job-2-import-impact-store/job-2-import-impact-store.job.ts	คลาส ImportImpactStoreJob — run(ctx) เรียงตาม flow ของ Job 2 ที่ละขั้น, ครอบ transaction, จบด้วย structured log
src/batch/sbpgi/job-2-import-impact-store/job-2-import-impact-store.service.ts	คลาส ImportImpactStoreService — logic ต่อชั้น (อ่าน/parse/คำนวณ/เขียน) + repository token ที่ inject จาก DATA_SOURCE
src/batch/sbpgi/job-2-import-impact-store/job-2-import-impact-store.config.ts	คลาส SbpgiJob2Config (แบบเดียวกับ src/config/app.config.ts — โปรเจกต์นี้ไม่ใช่ registerAs) — cron และพารามิเตอร์ทั้ง 6 ตัวของ Job 2 อ่านจาก env/config file (ไม่มีตาราง job_configs)
src/batch/sbpgi/job-2-import-impact-store/job-2-import-impact-store.module.ts	NestJS module ผูก job + service + repository provider (factory token string) เข้ากับ DatabaseModule
src/batch/runner.ts	ตัวรันกลาง: resolve job ตาม jobNo, กันรันซ้อนด้วย advisory lock, จับ error → แจ้งเตือน, เขียน structured log สรุป (ใช้รวมทั้ง 11 job)
src/batch/scheduler.ts	ลงทะเบียน cron จาก config (SBPGI_JOB2_CRON = 0 07 7 * *) และรองรับสั่งรันนอกรอบผ่าน CLI/runbook
src/batch/job-failure.notifier.ts	ส่งอีเมลแจ้งผู้ดูแลเมื่อ job ล้มเหลว ผ่าน EmailLibService ของ @gosoft-sbp/email-lib (log ลง email_sent ให้อัตโนมัติ)

9.2 Config Schema ของ Job 2 (backend config / env)

cron ปัจจุบันของ Job 2 คือ 0 07 7 * * (ทุกวันที่ 7 เวลา 07:00) — ประกาศเป็น SBPGI_JOB2_CRON และอ่านตอน bootstrap ของ scheduler.ts; ถ้า enabled=false scheduler ต้องไม่ลงทะเบียน cron ของ job นี้

```
// src/batch/sbpgi/job-2-import-impact-store/job-2-import-impact-store.config.ts
// convention จริ่งของ store-backend คือคลาส config ('src/config/app.config.ts' ที่ export ผ่าน
// 'AppConfigModule' แบบ @Global แล้วอ่าน process.env ตรงๆ) — โปรเจกต์นี้ **ไม่ได้ใช้ registerAs**
// แม้แต่จุดเดียว จึงประกาศเป็นคลาสให้รีวิว/ทดสอบเหมือน config ตัวอื่น
import { Injectable } from '@nestjs/common';

// TODO: Job 2 ไม่มีตาราง job_configs และไม่มี Job Admin API แล้ว (ตัดสินใจ 2026-08-06)
// TODO: ค่าทุกตัวอ่านจาก env/config file ของ backend เท่านั้น — เปลี่ยนค่า = แก่ config แล้ว deploy
export interface Job2Config {
  /** เปิด/ปิด job รอบถัดไปโดยไม่ต้อง deploy โค้ด */
  enabled: boolean;
  /** cron ของ job นี้ (อ่านตอน bootstrap ของ scheduler.ts) */
  cron: string;
  /** กำหนดการรัน (Cron) — ทุกวันที่ 7 ของเดือน เวลา 07:00 */
  cron: string;
  /** Argument (ขอบเขต/งวด) — รูปแบบ ZONES|YYYY|MM หรือ ALL|YYYY|MM — ไม่ระบุจะใช้งวดตาม modifyDateToString: □□ ปีในตัวอย่างเป็น
  พ.ศ. (2569) ตามค่าที่ระบบเดิมใช้กับวิว ALLMAP ซึ่งขัดกับกติกา ค.ศ. ทั้งระบบ (มติ 2026-08-06) — ต้องยืนยันกับเจ้าของ ALLMAP ว่าวิวเก็บปีเป็น พ.ศ.
  จริงหรือไม่ ถ้าไม่ ให้เปลี่ยนเป็น ค.ศ. (2026) */
  argument: string;
  /** Source View — dedup ด้วย ROW_NUMBER */
  sourceView: string;
  /** Branch Type ที่เข้าเกณฑ์ — FPT1 เข้าเกณฑ์เฉพาะเมื่อ SBP_CANCEL_TYPE_I = 06 */
  branchType: string;
  /** กฎ DENY (ตรวจสอบ ON_PROCESS) */
  denyOnProcess: string;
  /** PK Sequence */
  pkSequence: string;
  /** ผู้รับอีเมลเมื่อ job ล้มเหลว — เก็บเป็น string คั่น comma ให้ตรง signature ของ
  `EmailLibService.sendMail({ mailTo })` ที่รับ string ไม่ใช่ string[] */
  mailTo: string;
}
```

```

@Injectable()
export class SbpqiJob2Config implements Job2Config {
  // TODO: ยืนยันค่า default ทุกตัวกับ Ops ก่อนขึ้น production (ไม่มีหน้าจอบอกค่าแล้ว)
  enabled = (process.env.SBPQI_JOB2_ENABLED ?? 'true') === 'true';
  cron = process.env.SBPQI_JOB2_CRON ?? '0 07 7 * *';
  cron = process.env.SBPQI_JOB2_CRON ?? '0 07 7 * *'; // TODO: แก่ผ่าน env/config file แล้ว deploy
  argument = process.env.SBPQI_JOB2_ARGUMENT ?? 'ALL[2569]06'; // TODO: ปีในตัวอย่างเป็น พ.ศ. (2569) ตามค่าที่ระบบเดิมใช้กับวิว ALLMAP
  ซึ่งขัดกับกติกา ค.ศ. ทั้งระบบ (มติ 2026-08-06) — ต้องยืนยันกับเจ้าของ ALLMAP ว่าวิวเก็บปีเป็น พ.ศ. จริงหรือไม่ ถ้าไม่ ให้เปลี่ยนเป็น ค.ศ. (2026) (□□)
  sourceView = process.env.SBPQI_JOB2_SOURCE_VIEW ?? 'allmapssa.SEVEN_IMPACT_VIEW (SQL Server GSMALLMAP)'; // TODO:
  ค่าคงที่ทางธุรกิจ — เปลี่ยนต้องผ่านการอนุมัติ
  branchType = process.env.SBPQI_JOB2_BRANCH_TYPE ?? 'B, FAM, FB1, FB2, FC1, FVB, FVC, FPT1'; // TODO: ค่าคงที่ทางธุรกิจ —
  เปลี่ยนต้องผ่านการอนุมัติ
  denyOnProcess = process.env.SBPQI_JOB2_DENY_ON_PROCESS ?? 'สาขา N=F / juristic เดียวกัน / สัญญาไม่คลุมงวด / เก่ากว่า 12 เดือน'; //
  TODO: ค่าคงที่ทางธุรกิจ — เปลี่ยนต้องผ่านการอนุมัติ
  pkSequence = process.env.SBPQI_JOB2_PK_SEQUENCE ?? 'BIGSERIAL ของ fgi_impact_stores (PostgreSQL — ไม่ใช่ named sequence
  แบบ Oracle)'; // TODO: ค่าคงที่ทางธุรกิจ — เปลี่ยนต้องผ่านการอนุมัติ
  mailTo = process.env.SBPQI_JOB2_MAIL_TO ?? ''; // TODO: ผู้รับอีเมลแจ้ง error คั่นด้วย comma (เดิม: go-sbp (hardcoded, template 34))
}

// TODO: เพิ่ม SbpqiJob2Config ใน providers/exports ของ AppConfigModule (@Global) เหมือน AppConfig

```

9.3 Job Class — `run(ctx)` ของ Job 2 ที่จะขึ้นตามผัง

9.3.1 สัญญาของชั้นกลาง (`runner.ts`) + โครง service ของ Job 2

job class อ้าง JobRunContext / JobRunResult / JobState / JobFailedError — ทั้งหมดนิยาม ครั้งเดียวใน src/batch/runner.ts (ไฟล์รวมของทุก job ให้ merge ไม่ใช่เขียนทับ) และ service ต้องมี method ครบตามตารางขั้นตอนด้านล่าง มิฉะนั้น job class จะเรียก method ที่ไม่มีอยู่

```

// src/batch/runner.ts — สัญญากลางของทุก job (ประกาศครั้งเดียว ใช้ร่วมทั้ง 10 ฉบับ)

export interface JobRunContext {
  jobNo: string;
  period: string; // YYYYMM ของงวดที่รัน
  triggeredBy: string; // 'CRON' | userId ที่สั่งรันนอกรอบ
  params?: Record<string, string>;
}

export interface JobRunResult {
  event: 'job.finish';
  jobNo: string;
  jobName: string;
  status: 'SUCCESS' | 'SKIPPED' | 'SKIPPED_LOCKED' | 'FAILED';
  period: string;
  output: string;
  read: number; written: number; skipped: number; rejected: number;
  durationMs: number;
}

/** counter + ค่าที่ทุกชั้นของ job ใช้ร่วมกัน (service เป็นผู้สร้างผ่าน createState) */
export interface JobState {
  period: string;
  read: number; written: number; skipped: number; rejected: number;
  // TODO: เพิ่ม field เฉพาะของ job นี้ (เช่น rows ที่อ่านมา, path ไฟล์ที่เขียน)
  [key: string]: unknown;
}

/** error ที่ทำให้ job จบเป็น FAILED และส่งอีเมลแจ้งผู้ดูแล */
export class JobFailedError extends Error {
  constructor(public readonly code: string, message: string) { super(message); }
}

/** ให้ออกจาก transaction เมื่อสาขา NO บอกให้ข้ามงวด/เรคคอร์ด — runner สรุปรูปเป็น SKIPPED ไม่ใช่ FAILED */
export class JobSkippedError extends Error {}

// ImportImpactStoreService — method ที่ job class เรียก (1 method ต่อ 1 ชั้นในตารางด้านบน)
import { Inject, Injectable } from '@nestjs/common';
import type { DataSource, EntityManager } from 'typeorm';
import type { JobRunContext, JobState } from '../runner';
export type { JobState };

@Injectable()
export class ImportImpactStoreService {
  constructor(@Inject('DATA_SOURCE') private readonly dataSource: DataSource) {}

  createState(ctx: JobRunContext): JobState {
    return { period: ctx.period, read: 0, written: 0, skipped: 0, rejected: 0 };
  }

  // อ่าน SEVEN_IMPACT_VIEW จาก ALLMAP (ROW_NUMBER dedup)
  async step02Read(state: JobState, manager?: EntityManager): Promise<void> {
    // TODO: implement
  }
}

```

```

}

// มีข้อมูลต้นทาง?
async check03Condition(state: JobState): Promise<boolean> {
  return true; // TODO: เงื่อนไขจริงตามผัง
}

// เป็นคู่ร้านใหม่ (ยังไม่มีใน Oracle)?
async check04Update(state: JobState): Promise<boolean> {
  return true; // TODO: เงื่อนไขจริงตามผัง
}

// insert คู่ใหม่ sales_request_status = W
async step05Insert(state: JobState, manager?: EntityManager): Promise<void> {
  // TODO: implement
}

// เติมข้อมูล master และ enrichment data
async step06Enrich(state: JobState, manager?: EntityManager): Promise<void> {
  // TODO: implement
}

// ผ่านกฎ DENY? (ตรวจก่อน ON_PROCESS)
async check07Validate(state: JobState): Promise<boolean> {
  return true; // TODO: เงื่อนไขจริงตามผัง
}

// เข้าเงื่อนไข ON_PROCESS หรือ fgi_impact_processes.datasource = STA?
async check08Condition(state: JobState): Promise<boolean> {
  return true; // TODO: เงื่อนไขจริงตามผัง
}

// sales_request_status = P (On Process) แล้ววนจนครบทุกแถว
async step09Process(state: JobState, manager?: EntityManager): Promise<void> {
  // TODO: implement
}
}

```

9.3.2 `run(ctx)` ของ Job 2

ทุกชั้นใน `run()` ตรงกับ flowchart ของ Job 2 หนึ่งต่อหนึ่ง (decision และ error path รวมอยู่ด้วย) — method ที่ต้อง implement ใน service ตามตารางนี้

ลำดับ	ชนิด	ขั้นตอนจากผัง	Method ที่ต้อง implement	เส้นทาง NO / error
1	start	เริ่ม	<code>createState()</code>	-
2	io	อ่าน SEVEN_IMPACT_VIEW จาก ALLMAP (ROW_NUMBER dedup)	<code>step02Read()</code>	throw <code>JobFailedError</code> เมื่อทำไม่สำเร็จ
3	decision	มีข้อมูลต้นทาง?	<code>check03Condition()</code>	[end] จบการทำงาน
4	decision	เป็นคู่ร้านใหม่ (ยังไม่มีใน Oracle)?	<code>check04Update()</code>	[branch] ข้ามรายการ — ของเดิมไม่ถูกอัปเดต (updateList เป็น dead code)
5	process	insert คู่ใหม่ sales_request_status = W	<code>step05Insert()</code>	throw <code>JobFailedError</code> เมื่อทำไม่สำเร็จ
6	process	เติมข้อมูล master และ enrichment data	<code>step06Enrich()</code>	throw <code>JobFailedError</code> เมื่อทำไม่สำเร็จ
7	decision	ผ่านกฎ DENY? (ตรวจก่อน ON_PROCESS)	<code>check07Validate()</code>	[err] sales_request_status = N (Deny)
8	decision	เข้าเงื่อนไข ON_PROCESS หรือ fgi_impact_processes.datasource = STA?	<code>check08Condition()</code>	[branch] คงค่า W (รอตรวจสอบ)
9	process	sales_request_status = P (On Process) แล้ววนจนครบทุกแถว	<code>step09Process()</code>	throw <code>JobFailedError</code> เมื่อทำไม่สำเร็จ
10	end	จบ	<code>summarize()</code>	-


```

// src/batch/sbpgi/job-2-import-impact-store/job-2-import-impact-store.job.ts
import { Inject, Injectable, Logger } from '@nestjs/common';
import type { DataSource, EntityManager } from 'typeorm';
import { ImportImpactStoreService, type JobState } from './job-2-import-impact-store.service';
// 4 symbol นี้ใช้ใน src/batch/runner.ts (ดูหัวข้อ 9.3.1)
import { JobFailedError, JobSkippedError, JobRunContext, JobRunResult } from '../runner';

@Injectable()
export class ImportImpactStoreJob {
  static readonly jobNo = '2';
  private readonly logger = new Logger(ImportImpactStoreJob.name);

  constructor(
    // TODO: DATA_SOURCE = custom provider ที่ route SELECT/WITH ไป slave pool และ write ไป master
    @Inject('DATA_SOURCE') private readonly dataSource: DataSource,
    private readonly service: ImportImpactStoreService,
  ) {}

  async run(ctx: JobRunContext): Promise<JobRunResult> {
    const startedAt = Date.now();
    // TODO: state คือ counter (read/written/skipped/rejected) และค่าจาก job2Config
    const state = this.service.createState(ctx);
    try {
      // ขั้นที่ 2: อ่าน SEVEN_IMPACT_VIEW จาก ALLMAP (ROW_NUMBER dedup) · TODO: เชื่อม SQL Server GSMALLMAP ด้วย user allmapssa
      await this.service.step02Read(state);
      // ขั้นที่ 3 (decision): มีข้อมูลต้นทาง?
      const ok03 = await this.service.check03Condition(state);
      if (!ok03) { // NO → จบการทำงาน
        return this.summarize(state, 'SKIPPED', startedAt);
      }
      // ขั้นที่ 4 (decision): เป็นคู่ร้านใหม่ (ยังไม่มีใน Oracle)? · TODO: Errata E4: รันซ้ำจะไม่อัปเดตคู่เดิม
      const ok04 = await this.service.check04Update(state);
      if (!ok04) { // NO → ข้ามรายการ — ของเดิมไม่ถูกอัปเดต (updateList เป็น dead code)
        // TODO: เส้น NO ของขั้นนี้เป็น branch ระดับ record — ฟังไม่ได้ระบุว่าหยุดหรือไปต่อ
        // ถ้าเป็น 'ข้ามรายการ' -> state.skipped += 1; แล้ว continue ในลูปของ record
        // ถ้าเป็น 'ตั้งค่าแล้วไปต่อ' -> เรียก service ตั้งค่าสถานะ แล้วเดินขั้นถัดไป (ห้าม return)
        // ถ้าเป็น 'คงสถานะเดิม/ไม่เปิดงาน' -> หยุดเฉพาะ record นี้ ห้ามไหลไปขั้นถัดไป
      }
      // === transaction boundary === TODO: หนึ่ง transaction + savepoint
      await this.dataSource.transaction(async (manager: EntityManager) => {
        // ขั้นที่ 5: insert คู่ใหม่ sales_request_status = W · TODO: ช่องทางต้นทางเก็บที่ fgi_impact_processes.datasource = ALM (fgi_impact_stores
        // ไม่มีคอลัมน์ created_by/datasource)
        await this.service.step05Insert(state, manager);
      });
      // ขั้นที่ 6: เพิ่มข้อมูล master และ enrichment data · TODO: INNER JOIN — ถ้า master ไม่ครบ แฉจะหลุดหายเจียบ ๆ
      await this.service.step06Enrich(state);
      // ขั้นที่ 7 (decision): ผ่านกฎ DENY? (ตรวจก่อน ON_PROCESS) · TODO: DENY: สาขา N=F / juristic เดียวกัน / สัญญา SBP ไม่คลุมงวด / เก่ากว่า
      // 12 เดือน
      const ok07 = await this.service.check07Validate(state);
      if (!ok07) throw new JobFailedError('JOB2_STEP07', 'sales_request_status = N (Deny)');
      // ขั้นที่ 8 (decision): เช้าเงื่อนไข ON_PROCESS หรือ fgi_impact_processes.datasource = STA? · TODO: แห้ง STA เข้าสถานะ P ได้อัตโนมัติ
      const ok08 = await this.service.check08Condition(state);
      if (!ok08) { // NO → คงค่า W (รอตรวจสอบ)
        // TODO: เส้น NO ของขั้นนี้เป็น branch ระดับ record — ฟังไม่ได้ระบุว่าหยุดหรือไปต่อ
        // ถ้าเป็น 'ข้ามรายการ' -> state.skipped += 1; แล้ว continue ในลูปของ record
        // ถ้าเป็น 'ตั้งค่าแล้วไปต่อ' -> เรียก service ตั้งค่าสถานะ แล้วเดินขั้นถัดไป (ห้าม return)
        // ถ้าเป็น 'คงสถานะเดิม/ไม่เปิดงาน' -> หยุดเฉพาะ record นี้ ห้ามไหลไปขั้นถัดไป
      }
      // ขั้นที่ 9: sales_request_status = P (On Process) แล้ววนจนครบทุกแถว
      await this.service.step09Process(state);
      return this.summarize(state, 'SUCCESS', startedAt);
    } catch (error) {
      // TODO: error path ของ Job 2 — E4: updateList เป็น dead code / INNER JOIN ทำแถวที่ master ไม่ครบหายเจียบ (P1)
      this.logger.error(JSON.stringify({ event: 'job.failed', jobNo: '2', period: ctx.period,
        triggeredBy: ctx.triggeredBy, durationMs: Date.now() - startedAt, error: (error as Error).message }));
      // TODO: แจ้งผู้ดูแลผ่าน JobFailureNotifier (หัวข้อ 9.6.1) — runner เป็นผู้เรียกให้
      throw error;
    }
  }

  private summarize(state: JobState, status: JobRunResult['status'], startedAt = Date.now()): JobRunResult {
    // TODO: structured log บรรทัดเดียวจบ — ไม่มีตาราง job_run_histories แล้ว (2026-08-06)
    const summary = {
      event: 'job.finish', jobNo: '2', jobName: 'ImportImpactStore', status,
      period: state.period, output: 'fgi_impact_stores',
      read: state.read, written: state.written, skipped: state.skipped,
      rejected: state.rejected, durationMs: Date.now() - startedAt,
    };
    this.logger.log(JSON.stringify(summary));
    return summary as JobRunResult;
  }
}

```

9.4 การกันรันซ้อนของ Job 2 (PostgreSQL advisory lock)

Job 2 มีข้อควรระวังจาก legacy: E4: updateList เป็น dead code / INNER JOIN ทำแถวที่ master ไม่ครบหายเจียบ (P1) — runner ล็อกด้วย pg_try_advisory_lock ก่อนเริ่มรันแรกเสมอ และรอบที่ล็อกไม่ได้ให้จบด้วยสถานะ SKIPPED_LOCKED (ไม่ใช่ FAILED)

```
// src/batch/runner.ts (ส่วนกันรันซ้อน)
import { Inject, Injectable, Logger } from '@nestjs/common';
import type { DataSource } from 'typeorm';

// TODO: ห้ามใช้แถวสถานะ RUNNING ในตารางเป็นตัวกัน (ไม่มีตาราง job_run_histories แล้ว)
// ใช้ PostgreSQL advisory lock ระดับ session แทน — ปลดล็อกอัตโนมัติเมื่อ connection หลุด
export const SBPGI_JOB_LOCK_CLASS_ID = 861000; // namespace ของระบบ SBPGI
export const JOB_LOCK_KEYS: Record<string, number> = { '2': 20 /* TODO: เพิ่มครบทั้ง 11 job */ };

@Injectable()
export class BatchRunner {
  private readonly logger = new Logger(BatchRunner.name);
  constructor(@Inject('DATA_SOURCE') private readonly dataSource: DataSource) {}

  async runExclusive<T>(jobNo: string, fn: () => Promise<T>): Promise<T | { status: 'SKIPPED_LOCKED' }> {
    // TODO: ต้องใช้ QueryRunner (connection เดียวบน master) — dataSource.query() ของโปรเจกต์นี้
    // route SQL ที่ขึ้นต้นด้วย SELECT ไป slave pool ทำให้ lock ไปตกที่ replica คนละ connection
    const runner = this.dataSource.createQueryRunner('master');
    await runner.connect();
    const objectId = JOB_LOCK_KEYS[jobNo];
    try {
      const [{ locked }] = await runner.query(
        'SELECT pg_try_advisory_lock($1, $2) AS locked',
        [SBPGI_JOB_LOCK_CLASS_ID, objectId],
      );
      if (!locked) {
        // TODO: รอบนี้ข้ามไปเลย ๆ ไม่ถือเป็น error และไม่ต้องส่งอีเมล
        this.logger.warn(JSON.stringify({ event: 'job.skipped.locked', jobNo }));
        return { status: 'SKIPPED_LOCKED' };
      }
      return await fn();
    } finally {
      // TODO: ปลด lock ทุกกรณี แล้วคืน connection เข้า pool
      await runner.query('SELECT pg_advisory_unlock($1, $2)', [SBPGI_JOB_LOCK_CLASS_ID, objectId]);
      await runner.release();
    }
  }
}
```

9.5 Repository / SQL หลักของ Job 2

repository ของ Job 2 ประกาศเป็น factory provider ({provide: 'IMPORT_IMPACT_STORE_REPOSITORY', useFactory: (ds) => ds.getRepository(Entity), inject: ['DATA_SOURCE']}) แล้วยิง raw SQL ตามแบบ module ธุรกิจอื่นของ store-backend (schema sps_store มาจาก search_path)

ตาราง	R/W	การใช้งานตามผัง	หมายเหตุ target design
fgi_impact_stores	W	insert คู่ร้านกระทบ-ร้านใหม่ / ตั้ง sales_request_status = W · N · P (ตารางนี้ไม่มี created_by — ช่องทางต้นทางอยู่ที่ fgi_impact_processes.datasource)	เขียน SQL ตรงผ่าน DATA_SOURCE

```
-- Job 2 ImportImpactStore — query หลักที่ต้อง implement
-- TODO: ทุก statement รันผ่าน DATA_SOURCE (SELECT ไป slave, write ไป master) และ
-- write ทั้งหมดต้องอยู่ใน transaction เดียวกันที่ระบุใน 9.3

-- [W] fgi_impact_stores : insert คู่ร้านกระทบ-ร้านใหม่ / ตั้ง sales_request_status = W · N · P (ตารางนี้ไม่มี created_by — ช่องทางต้นทางอยู่ที่ fgi_impact_processes.datasource)
-- TODO: เพิ่มคอลัมน์ payload จริงจาก Database design
INSERT INTO fgi_impact_stores
  (* TODO: business key + payload + created_by, created_at */)
VALUES (* TODO: bind params ตามลำดับคอลัมน์ด้านบน */)
ON CONFLICT (impacted_store_code, new_store_code, impact_month) -- unique key จริงตาม DDL ของ fgi_impact_stores (ห้ามเดา)
DO UPDATE SET (* TODO: คอลัมน์ที่ยอมให้ทับ *)
  updated_at = NOW(), updated_by = 'JOB2';
```

9.6 การแจ้งเตือนและการรันซ้ำของ Job 2

9.6.1 อีเมลแจ้งผู้ดูแลเมื่อ job ล้มเหลว

ใช้ EmailLibService จาก @gosoft-sbp/email-lib ตัวเดียวกับที่ระบบเดิมใช้ (inform-evaluate / external-audit / statement PTT) — ไม่สร้างกลไกส่งเมลใหม่

```
// src/batch/job-failure.notifier.ts
import { Injectable, Logger } from '@nestjs/common';
// ชื่อ method ของ lib ที่ store-backend เรียกจริงคือ `sendMail` (ไม่ใช่ `sendEmail`) และ
// `mailTo` / `mailCc` เป็น **string** คั่นด้วย comma — ดู evaluation-process.service.ts,
// external-audit.service.ts, statement.service.ts, inform-evaluate.service.ts, performance.service.ts
import { EmailLibService } from '@gosoft-sbp/email-lib';
import type { JobRunContext } from './runner';

@Injectable()
export class JobFailureNotifier {
  private readonly logger = new Logger(JobFailureNotifier.name);
  // TODO: ใช้ lib อีเมลของระบบเดิม — template อยู่ในตาราง email_template และ log ลง email_sent อัตโนมัติ
  // (ตั้งชื่อ property ว่า mailService ตาม call site เดิมทุกที่ใน store-backend)
  constructor(private readonly mailService: EmailLibService) {}

  async notifyFailure(jobNo: string, ctx: JobRunContext, error: Error): Promise<void> {
    // TODO: ผู้รับของ Job 2 เดิมคือ go-sbp (hardcoded, template 34) — ย้ายมาเป็น env SBPGI_JOB2_MAIL_TO
    const recipients = (process.env.SBPGI_JOB2_MAIL_TO ?? '').split(',').map((s) => s.trim()).filter(Boolean);
    if (!recipients.length) {
      this.logger.warn(JSON.stringify({ event: 'job.mail.skipped', jobNo, reason: 'NO_RECIPIENT' }));
      return;
    }
    try {
      await this.mailService.sendMail({
        // TODO: emailId = id ของ template EM-07 (แจ้ง error batch) ในตาราง email_template
        emailId: Number(process.env.SBPGI_JOB_FAIL_EMAIL_TEMPLATE_ID),
        mailTo: recipients.join(','), // signature รับ string ไม่ใช่ string[]
        mailCc: '',
        param: {
          jobNo, jobName: 'ImportImpactStore',
          jobTitle: 'นำเข้าฐานถูกกระทบจาก ALLMAP',
          period: ctx.period, triggeredBy: ctx.triggeredBy,
          output: 'fgi_impact_stores',
          errorMessage: error.message,
          rerunNote: 'คู่มือถูกข้าม — รันซ้ำไม่อัปเดตของเดิม ต้องลบ/แก้คู่มือที่ต้องการอย่างจริงจังก่อน',
        },
      });
    } catch (mailError) {
      // TODO: ส่งเมลไม่สำเร็จห้ามกลับ error เดิมของ job — log แล้วปล่อยผ่าน
      this.logger.error(JSON.stringify({ event: 'job.mail.failed', jobNo, error: (mailError as Error).message }));
    }
  }
}
```

9.6.2 Checklist การ rerun

- กติกา rerun ของ Job 2: คู่มือถูกข้าม — รันซ้ำไม่อัปเดตของเดิม ต้องลบ/แก้คู่มือที่ต้องการอย่างจริงจังก่อน
- ขอบเขต transaction ที่ต้องรักษามือรันซ้ำ: หนึ่ง transaction + savepoint
- ความเสี่ยงที่ต้องตรวจก่อน/หลังรันซ้ำ: E4: updateList เป็น dead code / INNER JOIN ทำแถวที่ master ไม่ครบหายเงียบ (P1)
- ตรวจสอบว่ารอบก่อนหน้าไม่ได้ค้าง lock อยู่ (SELECT * FROM pg_locks WHERE locktype = 'advisory') ก่อนสั่งรันนอกรอบ
- สั่งรันนอกรอบผ่าน CLI/runbook เท่านั้น (ไม่มีหน้าจอและไม่มี Job Admin API): node dist/batch/cli.js --job=2 --period=<YYYYMM>
- หลังรันซ้ำ ตรวจสอบ output fgi_impact_stores และ log บรรทัด job.finish ว่า read/written/skipped/rejected ตรงกับที่คาด
- ถ้ารอบก่อนล้มเหลวกลางทาง ตรวจสอบ interface_transactions ของงวดนั้นว่ามีแถวค้างสถานะ READY/PENDING หรือไม่ ก่อนสั่งรันใหม่

10. Processing Flow

Step	Description
1	เริ่ม
2	อ่าน SEVEN_IMPACT_VIEW จาก ALLMAP (ROW_NUMBER dedup) (เชื่อม SQL Server GSMALLMAP ด้วย user allmapssa)
3	มีข้อมูลต้นทาง? No: จบการทำงาน

Step	Description
4	เป็นคู่ร้านใหม่ (ยังไม่มีใน Oracle)? No: ข้ามรายการ — ของเดิมไม่ถูกอัปเดต (updateList เป็น dead code) (Errata E4: ร้านซ้ำจะไม่อัปเดตคู่เดิม)
5	insert คู่ใหม่ sales_request_status = W (ช่องทางต้นทางเก็บที่ fgi_impact_processes.datasource = ALM (fgi_impact_stores ไม่มีคอลัมน์ created_by/datasource))
6	เติมข้อมูล master และ enrichment data (INNER JOIN — ถ้า master ไม่ครบ แล้วยจะหลุดหายเจียบ ๆ)
7	ผ่านกฎ DENY? (ตรวจสอบก่อน ON_PROCESS) No: sales_request_status = N (Deny) (DENY: สาขา N=F / juristic เดียวกัน / สัญญา SBP ไม่คลุมงวด / เก่ากว่า 12 เดือน)
8	เข้าเงื่อนไข ON_PROCESS หรือ fgi_impact_processes.datasource = STA? No: คงค่า W (รอตรวจสอบ) (แหล่ง STA เข้าสถานะ P ได้อัตโนมัติ)
9	sales_request_status = P (On Process) แล้ววนจนครบทุกแถว
10	จบ

11. Acceptance Criteria

- พารามิเตอร์และ cron อ่านจาก backend config เท่านั้น — เปลี่ยนค่าโดย deploy config ไม่ใช่ผ่าน API/หน้าจอ
- การรันต้องตรวจสอบ enabled flag ใน config และกันรันซ้อนด้วย distributed/advisory lock
- ทุกรอบต้องเขียน application log แบบ structured (เวลา/แถว/ไฟล์/ผล) และ error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

12. Developer Test Checklist

No	Test
1	รันตามตารางเวลาแล้วผลถูกต้องบน fixture
2	รันนอกรอบผ่าน CLI ได้ผลเดียวกับ cron
3	สั่งรันซ้อนขณะกำลังรัน → runner ปฏิเสธ (lock ทำงาน)
4	แก้ config แล้ว deploy → รอบถัดไปใช้ค่าใหม่
5	job throw error → EM-07 ออก และ log มีบรรทัด error
6	ตรวจสอบผลกระทบตารางตาม R/W mapping reference

13. Unit Test Scope

5 ชั่วโมง (30% ของ implementation 14 ชั่วโมง) · เครื่องมือ: Jest + mock repository/DataSource (ไม่ต้อง DB จริง)

หัวข้อนี้คือ unit test ที่ต้องเขียนคู่กับโค้ด — ต่างจาก *Developer Test Checklist* ซึ่งเป็น scenario ระดับ end-to-end/manual ที่ใช้ตอนตรวจรับ · รายการด้านล่าง derive จาก field/validation, acceptance criteria, endpoint และตารางที่เอกสารนี้เขียน

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
Branch Type ที่เข้าเกณฑ์	rule	ใช้กฎกับข้อมูลตัวอย่างแล้วได้ผลตามที่ระบุ — B, FAM, FB1, FB2, FC1, FVB, FVC, FPT1
กฎ DENY (ตรวจสอบก่อน ON_PROCESS)	rule	ใช้กฎกับข้อมูลตัวอย่างแล้วได้ผลตามที่ระบุ — สาขา N=F / juristic เดียวกัน / สัญญาไม่คลุมงวด / เก่ากว่า 12 เดือน
business rule	logic	พารามิเตอร์และ cron อ่านจาก backend config เท่านั้น — เปลี่ยนค่าโดย deploy config ไม่ใช่ผ่าน API/หน้าจอ

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
business rule	logic	การรันต้องตรวจ enabled flag ใน config และกันรันซ้อนด้วย distributed/advisory lock
business rule	logic	ทุกรอบต้องเขียน application log แบบ structured (เวลา/แถว/ไฟล์/ผล) และ error ต้องส่ง EM-07
business rule	logic	DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช้งานสร้างหน้า Database
business rule	logic	รองรับ rerun rule และ risk note ตาม runbook
fgi_impact_stores	transaction	จำลอง error กลางทาง แล้วยืนยันว่า rollback ครบ ไม่เหลือแถวค้าง (mock DataSource/QueryRunner)
runner	idempotency	รันซ้ำด้วย fixture เดิมต้องไม่เกิดแถวซ้ำ (ON CONFLICT / business unique key ทำงาน)
runner	lock	เรียกซ้อนขณะกำลังรัน ต้องถูกปฏิเสธด้วย advisory lock

- ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง — mock ที่ขอบ repository/client เสมอ
- ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่
- เกณฑ์ผ่านของ CI: ทุกเคสในตารางนี้มี test จริงและผ่านทั้งหมด

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. ImportImpactStore.java — lines 24-35

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ImportImpactStore.java
Original lines	24-35
Responsibility	Legacy main entrypoint for impacted-store import.

```
public class ImportImpactStore {

    private static ApplicationContext context = new FileSystemXmlApplicationContext("config.xml");
    private static FgiConfig config = (FgiConfig) context.getBean("fgiConfig");
    private static ImpactStoreService storeImpactService = (ImpactStoreService) context.getBean("storeImpactService");
        public static Locale LOCALE_EN = Locale.US;
        public static String DATE_FORMAT_DDMMYYYY_TIME = "dd/MM/yyyy HH:mm:ss";

    public static void main(String[] args) {
        LogUtils.info(ImportImpactStore.class, "=====***** Start => import FGI_IMPACT_STORE *****");
        Calendar cal = Calendar.getInstance(Locale.US);
        String year = th.co.gosoft.fcs.utils.DateUtils.modifyDateToString( new Date() , -1 , "yyyy" , LOCALE_EN );
```

J1. ImportImpactStore.java — lines 175-186

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ImportImpactStore.java
Original lines	175-186
Responsibility	Legacy main entrypoint for impacted-store import.

```

email.send();
LogUtils.info(ImportImpactStore.class,"---- send email : [success] ----");
statusSend = true ;
}catch(Exception e){
statusSend = false ;
LogUtils.error(ImportImpactStore.class,e);
LogUtils.info(ImportImpactStore.class,"---- send email : [fail] ----");
}
return statusSend ;
}
}

```

J2. ImportStoreJdbc.java — lines 30-41

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportStoreJdbc.java
Original lines	30-41
Responsibility	Query SEVEN_IMPACT_VIEW and insert/update FGI impact/new-store records.

```

public List<ImpactStore> selectVsevenImpact(int year, int month,String[] zone) {
String zoneStr = new String();
try{
List params = new LinkedList();
String sql = " SELECT * ";
sql += " FROM ( ";
sql += " SELECT *, ";
sql += " ROW_NUMBER() OVER(PARTITION BY STORECODE_I, STORECODE_N, PERIOD_YEAR, PERIOD_MONTH ORDER BY PERIOD_YEAR, PERIOD_MONTH) AS ROW_NUM ";
sql += " FROM " +FgiConstant.ALLMAP_SCHEMA_PRD +" SEVEN_IMPACT_VIEW ";
sql += " WHERE PERIOD_MONTH = ? AND PERIOD_YEAR = ? ";
if(zone != null && zone.length >0){
zoneStr = TextUtils.arrayElementToStringHasQuote(zone);
}
}
}

```

J2. ImportStoreJdbc.java — lines 73-84

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportStoreJdbc.java
Original lines	73-84
Responsibility	Query SEVEN_IMPACT_VIEW and insert/update FGI impact/new-store records.

```
sql.append(" SEQ_FGI_IMPACT_STORE.NEXTVAL, ?, ?, ");
sql.append(" ?, ?, ?, ?, ?, ? ");
sql.append(" ?, ?, ?, ?, ?, ? ");
sql.append(" ?, ?, ?, ?, ?, ? ");
sql.append(" ?, ? ");
sql.append(" ) ");
int[] result = jdbcTemplate.batchUpdate(sql.toString(), new ImportImpactStoreBatch(impactStoreList));
status = true;
LogUtils.info(getClass(), "----- insert FGI_IMPACT_STORE : " + result.length + " -----");

return status;
}
```


J2. ImportStoreJdbc.java — lines 170-181

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportStoreJdbc.java
Original lines	170-181
Responsibility	Query SEVEN_IMPACT_VIEW and insert/update FGI impact/new-store records.

```
public List<ImpactStore> getImpactStoreFranchise(int year, int month,String[] zoneArr) {
    StringBuffer sql = new StringBuffer();
    String zoneStr = new String();
    List params = new LinkedList();
    try{
        sql.append(" select distinct fis.storecode_i, fis.storecode_n, ? as month, ? as year ");
        sql.append(" from fgi_impact_store fis ");
        sql.append(" left join fgi_impact_store_on_process op on op.flag_action in ('Y','W') ");
        sql.append(" and op.storecode_i = fis.storecode_i ");
        sql.append(" and to_date(? || ?, 'YYYYMM') ");
        sql.append(" between to_date(op.start_compensate_year || op.start_compensate_month, 'YYYYMM') ");
        sql.append(" and ( ");
```

J2. ImportStoreJdbc.java — lines 473-484

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportStoreJdbc.java
Original lines	473-484
Responsibility	Query SEVEN_IMPACT_VIEW and insert/update FGI impact/new-store records.

```
}catch(Exception e){
    LogUtils.error(getClass(),e);
    return jdbcTemplate.batchUpdate(sql.toString(), new UpdateImpactSaleFromIASBatch(impactStoreList));
}

public EmailTemplateBean getEmailTemplateByPk(String emailTemplateId) {

    StringBuffer sql = new StringBuffer();
    sql.append("SELECT SUBJECT_FORMAT, BODY_FORMAT ,SENDER FROM EMAIL_TEMPLATE ");

    if(StringUtils.isEmpty(emailTemplateId)){
```